



JavaScript

The language of the web — from browser tricks to full-stack power



What is JavaScript, History & Alternatives

■ **Story:** Imagine the web in 1995 was a static newspaper — you could only read it. Brendan Eich was hired by Netscape and, in just **10 days**, created a scripting language to make pages interactive. It was first called *Mocha*, then *LiveScript*, then renamed **JavaScript** — purely as a marketing move to ride Java's hype.

Key Facts

- **Created:** 1995 by Brendan Eich at Netscape
- **First names:** Mocha → LiveScript → JavaScript
- **Standardised:** ECMAScript (ECMA-262) by ECMA International
- **Paradigm:** Multi-paradigm — OOP, functional, event-driven
- **Runtime:** V8 (Chrome/Node), SpiderMonkey (Firefox)

Why JavaScript?

- Only language natively run by browsers
- Full-stack capability via Node.js
- Huge ecosystem (npm > 2 million packages)
- Non-blocking, event-driven & fast for I/O
- Single language for frontend + backend

Alternatives to JavaScript

Language	Compiled to	Use Case
TypeScript	JS	Typed JS for large apps
CoffeeScript	JS	Cleaner syntax (legacy)
Dart	JS / native	Flutter, web apps
WebAssembly	Binary	Performance-critical tasks
Elm / ClojureScript	JS	Functional front-end

ECMAScript Evolution

- **ES5 (2009)** strict mode, JSON, Array methods
- **ES6 (2015)** let/const, arrow fn, classes, Promises, modules
- **ES2017** async/await
- **ES2020** Optional chaining (?.), nullish coalescing (??)
- **ES2022** Top-level await, class fields

Characteristics: Interpreted (JIT compiled), dynamically typed, single-threaded, prototype-based, first-class functions, garbage collected, event-driven.

✅ Advantages

- Runs in every browser
- Huge community & npm
- Non-blocking I/O
- Full-stack (Node.js)
- Quick to prototype

❌ Disadvantages

- Dynamically typed = runtime bugs
- Single-threaded (workarounds needed)
- Inconsistent browser behaviour (legacy)
- Security: client code is visible
- Callback hell (pre-Promises)

Variables & Declarations

Think of variables like labeled boxes in a warehouse. **var** is an old rusty box you can move anywhere (function scope). **let** is a proper box locked inside a room (block scope). **const** is a locked safe — the lock can't change, but items inside an object can still be rearranged.

Keyword	Scope	Hoisted?	Re-declare?	Re-assign?
var	Function	Yes (undefined)	Yes	Yes
let	Block	Yes (TDZ)	No	Yes
const	Block	Yes (TDZ)	No	No

```
var x = 10;           // function scoped
let y = 20;           // block scoped
const PI = 3.14;      // can't reassign

const obj = { a: 1 };
obj.a = 99;           //  allowed – obj reference is const, not contents
obj = {};             //  TypeError – can't reassign const binding
```

Copy

Scope, Scope Chain & Lexical Environment

Imagine nested rooms inside a house. If you lose your keys in the inner bedroom, you look there first, then in the hallway, then the living room, then outside. That search path is the **scope chain**. The blueprint of which variables belong to which room — created at code-writing time — is the **Lexical Environment**.

Types of Scope

- **Global scope** — accessible everywhere
- **Function scope** — inside a function only
- **Block scope** — inside `{ }` (let/const)
- **Module scope** — inside ES6 modules

Lexical Environment

Created at *write-time*. Contains:

- **Environment Record** — actual variables & functions
- **Outer reference** — link to parent scope

Scope chain is built by following these outer references.

```
let city = "Delhi";           // global

function country() {
  let state = "Rajasthan";    // function scope

  function village() {
    let area = "Jaipur";      // inner scope
    console.log(city);        //  reaches global via scope chain
    console.log(state);       //  reaches parent
    console.log(area);        //  own scope
  }
  village();
  // console.log(area);      //  ReferenceError
}
```

Copy

Hoisting & Temporal Dead Zone (TDZ)

Before your JavaScript code runs, the JS engine does a quick "pre-scan" — like a manager who walks through the office before work starts, noting where every desk (function/variable) will be, but not filling them yet. That pre-scan is hoisting. For **let/const**, the desk is noted but locked (TDZ) until the actual declaration line.

var hoisting

```
console.log(a); // undefined (hoisted)
var a = 5;
console.log(a); // 5
```

Copy

let/const — TDZ

```
console.log(b); // ❌ ReferenceError (TDZ)
let b = 10;
console.log(b); // 10
```

Copy

```
// Function declarations are fully hoisted
greet(); // ✅ Works!
function greet() { console.log("Hello"); }
```

Copy

```
// Function expressions are NOT
greet2(); // ❌ TypeError: greet2 is not a function
var greet2 = function() { console.log("Hi"); };
```

TDZ: The period between entering the block and the actual declaration line where **let/const** exist but cannot be accessed. Accessing them throws **ReferenceError**.

12 34 Primitive & Non-Primitive Data Types

📦 Primitives are like individual Lego bricks — they are values themselves, stored directly in the stack. Non-primitives (objects/arrays) are like a treasure map — the variable holds the *map (reference)*, not the treasure chest itself. That's why two variables pointing to the same object share the same chest!

Primitive (7 types)

- **string** — "hello"
- **number** — 42, 3.14, NaN, Infinity
- **boolean** — true / false
- **undefined** — declared but not assigned
- **null** — intentional empty value
- **bigint** — 9007199254740991n
- **symbol** — unique identifier

✅ Immutable · Stored by **value** · Copied on assignment

Non-Primitive (Reference types)

- **Object** — { key: value }
- **Array** — [1, 2, 3]
- **Function** — () ⇒ {}
- **Date** , **RegExp** , **Map** , **Set**

✅ Mutable · Stored by **reference** · Shared on assignment

```
let a = { x: 1 };
let b = a;           // same reference
b.x = 99;
console.log(a.x); // 99 ← a also changed!
```

Copy

🔒 Mutability & Immutability

📖 A string is like a printed book — you can't erase a letter; you have to print a whole new book with the change. An object is like a whiteboard — you can erase and rewrite as much as you like on the same board.

Immutable — Primitives

```
let name = "Ram";
name[0] = "S";    // silent fail in non-strict
console.log(name); // "Ram" — unchanged!
```

Copy

```
// Strings are immutable in JS.
// Every operation returns a NEW string.
let upper = name.toUpperCase(); // "RAM"
console.log(name); // still "Ram"
```

Mutable — Objects/Arrays

```
const arr = [1, 2, 3];
arr.push(4);           // ✅ mutates in place
arr[0] = 99;           // ✅ mutates
```

Copy

```
const obj = { a: 1 };
obj.b = 2;             // ✅ mutates
```

```
// To make object immutable:
Object.freeze(obj);
```

Yes — strings are immutable. You cannot change a character in place.
That is WHY `name[0] = "S"` silently does nothing.

```
obj.b = 99;           // silent fail
console.log(obj.b);  // 2
```

Type Conversion — Explicit & Implicit

Explicit (you do it)

```
Number("42")      // 42
Number(true)       // 1
Number(false)      // 0
Number(null)       // 0
Number(undefined) // NaN
Number("")         // 0

String(42)         // "42"
Boolean(0)         // false
Boolean("hi")      // true

parseInt("42px")   // 42
parseFloat("3.14x") // 3.14
```

 Copy

Implicit (JS does it)

```
"5" + 2           // "52" (+ triggers string concat)
"5" - 2           // 3   (- forces number)
"5" * "2"         // 10
true + 1          // 2
false + 1         // 1
null + 1          // 1
undefined + 1     // NaN

// == uses coercion:
0 == false // true ← dangerous!
"" == false // true
null == undefined // true

// === does NOT coerce:
0 === false // false ← safe
```

 Copy

Truthy & Falsy Values

Falsy (only 8)

```
false
0
-0
0n      // bigint zero
""      // empty string
null
undefined
NaN
```

 Copy

Truthy (everything else)

```
"0"           // truthy! non-empty string
"false"       // truthy!
[]            // truthy! empty array
{}            // truthy! empty object
function(){} // truthy
-1            // truthy
Infinity      // truthy
```

 Copy

Operators: Ternary, Short-Circuit, Nullish, Optional Chaining

Ternary vs if-else

```
// if-else — use for complex multi-line logic
if (age ≥ 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}

// Ternary — use for simple inline expression (returns a value)
const label = age ≥ 18 ? "Adult" : "Minor";

// ternary is an EXPRESSION (produces a value), if-else is a STATEMENT
```

 Copy

Short-Circuit Evaluation

```
// && – stops at first FALSY
false && doSomething(); // doSomething never called
true && console.log("yes"); // "yes"
const name = user && user.name; // safe access
```

[Copy](#)

```
// || – stops at first TRUTHY
const val = input || "default";
// if input is falsy, use "default"
```

[Copy](#)

```
// ?? – Nullish coalescing
// ONLY null/undefined triggers fallback (not 0 or "")
const count = input ?? 0;
// if input is null/undefined → 0
// if input is 0 → 0 (preserved!)
```

Optional Chaining vs Ternary

```
// Without optional chaining → error if user is null
console.log(user.address.city); // ❌ TypeError if user is null
```

[Copy](#)

```
// With optional chaining (?.):
console.log(user?.address?.city); // undefined (no error)
console.log(user?.getName?.()); // calls method safely
```

```
// Ternary checks ONE condition and returns one of two values.
// Optional chaining (?.) short-circuits the whole chain if null/undefined.
```

Types of Loops

Loop	Use For	Example
<code>for</code>	Known iterations	<code>for(let i=0;i<5;i++)</code>
<code>while</code>	Unknown iterations	<code>while(i<5)</code>
<code>do...while</code>	Run at least once	<code>do{...}while(i<5)</code>
<code>for...of</code>	Arrays, strings, iterables	<code>for(let x of arr)</code>
<code>for...in</code>	Object keys	<code>for(let k in obj)</code>
<code>forEach</code>	Array iteration (no return)	<code>arr.forEach(x ⇒ ...)</code>

forEach vs map

```
// forEach – iterates, returns undefined, CANNOT chain
[1,2,3].forEach(x ⇒ console.log(x * 2)); // side effects only
```

[Copy](#)

```
// map – iterates, returns NEW array, chainable
const doubled = [1,2,3].map(x ⇒ x * 2); // [2,4,6]
// forEach modifies world, map creates new world
```

Functions — All Types

Regular vs Arrow Functions

Feature	Regular	Arrow
<code>this</code>	Dynamic (caller)	Lexical (enclosing scope)
<code>arguments</code>	Available	Not available
Constructor	Can use <code>new</code>	Cannot use <code>new</code>

Feature	Regular	Arrow
Hoisted	Yes (declaration)	No

First-Class Functions

Functions are **first-class citizens** in JS — they can be assigned to variables, passed as arguments, returned from other functions, just like any other value.

Pure Functions

```
// Pure: same input → same output, no side effects
function add(a, b) { return a + b; } // ✅ pure

// Impure: depends on external state
let count = 0;
function increment() { count++; } // ❌ impure — side effect
```

Copy

Higher-Order Functions

```
// A function that takes/returns a function
function multiplier(factor) {
  return function(num) { return num * factor; };
}

const double = multiplier(2);
double(5); // 10

// map, filter, reduce are all HOFs!
```

Copy

call, apply, bind

```
function greet(greeting, punct) {
  return `${greeting}, ${this.name}${punct}`;
}

const user = { name: "Arjun" };

greet.call(user, "Hello", "!"); // "Hello, Arjun!" — args spread
greet.apply(user, ["Hi", "?"]); // "Hi, Arjun?" — args as array
const fn = greet.bind(user);    // returns new function with 'this' fixed
fn("Hey", ".");                 // "Hey, Arjun."
```

Copy

Variadic Functions & arguments

```
// Old way — arguments object (array-like, not real array)
function sum() {
  return [...arguments].reduce((a, b) => a + b, 0);
}

// Modern way — rest params (real array)
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}

sum(1, 2, 3, 4); // 10
```

Copy

Closures

Imagine you leave a sticky note on your fridge before going on vacation. Even when you're away (the function returned), the note is still there — the inner function remembers the variables of the outer function. That sticky note is a **closure**.

```
function makeCounter() {
  let count = 0;          // "sticky note"
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count
  };
}

const counter = makeCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.decrement(); // 1
counter.value();     // 1
// count is private! can't access from outside.
```

Copy

Closures enable: **data privacy**, **factory functions**, **memoization**, **module patterns**.

this Keyword

📌 **this** is like a name badge at a conference. In a panel (regular function), the badge says whoever is speaking (the caller). In an arrow function, you keep your own badge from home (the enclosing context). In strict mode, if no one is the host, the badge is blank (undefined).

Context	Value of this
Global scope (non-strict)	<code>window</code> (browser)
Global scope (strict)	<code>undefined</code>
Object method	The object itself
Arrow function	Inherits from enclosing scope
<code>new</code> constructor	Newly created object
<code>call/apply/bind</code>	Explicitly provided
Event listener	The DOM element

```
const obj = {
  name: "Priya",
  greet: function() { return this.name; },      // "Priya"
  greetArrow: () => { return this.name; }        // undefined (lexical this = global)
};

// Why use this? To write reusable methods that operate on the calling object.
```

Copy

Prototypes & Prototypal Inheritance

📌 Every object in JS has a hidden parent (prototype). If you ask an object for a property it doesn't have, it asks its parent, then grandparent — climbing the **prototype chain** until it hits `null`. Like inheriting family traits — no need to copy, just look up the family tree.

```
function Animal(name) {
  this.name = name;
}

Animal.prototype.speak = function() {
  return `${this.name} makes a sound.`;
};
```

Copy

```
const dog = new Animal("Dog");
dog.speak(); // "Dog makes a sound." – found on prototype, not dog itself

// Modern class syntax (syntactic sugar over prototypes)
class Vehicle {
  constructor(brand) { this.brand = brand; }
  honk() { return `${this.brand} goes beep!`; }
}
class Car extends Vehicle {
  drive() { return `${this.brand} drives.`; }
}
const c = new Car("Tesla");
c.honk(); // inherited from Vehicle.prototype
```

🔧 Built-in Methods — Array, Object, String, Math, Number

What is a method? A method is a *function* that belongs to an object (or prototype). When you call `arr.push()`, `push` is the method, `arr` is the object it operates on.

Array Methods

Method	What it does	Returns
<code>push(x)</code>	Add to end	New length
<code>pop()</code>	Remove from end	Removed item
<code>shift()</code>	Remove from start	Removed item
<code>unshift(x)</code>	Add to start	New length
<code>map(fn)</code>	Transform each item	New array
<code>filter(fn)</code>	Keep matching items	New array
<code>reduce(fn, init)</code>	Accumulate to one value	Single value
<code>find(fn)</code>	First matching item	Item or undefined
<code>findIndex(fn)</code>	Index of first match	Index or -1
<code>some(fn)</code>	Any item matches?	boolean
<code>every(fn)</code>	All items match?	boolean
<code>includes(x)</code>	Contains item?	boolean
<code>indexOf(x)</code>	First index of item	index or -1
<code>slice(s,e)</code>	Copy portion	New array
<code>splice(s,n, ...)</code>	Remove/insert items (mutates)	Removed items
<code>flat(depth)</code>	Flatten nested arrays	New array
<code>flatMap(fn)</code>	map + flat(1)	New array
<code>sort(fn)</code>	Sort in place	Same array
<code>reverse()</code>	Reverse in place	Same array
<code>join(sep)</code>	Array to string	String
<code>concat(...)</code>	Merge arrays	New array
<code>fill(val,s,e)</code>	Fill with value	Same array (mutates)

Method	What it does	Returns
<code>Array.from(x)</code>	Convert to array	New array
<code>Array.isArray(x)</code>	Is it an array?	boolean

Object Methods

```
const obj = { a: 1, b: 2, c: 3 };

Object.keys(obj)    // ["a","b","c"]
Object.values(obj)  // [1,2,3]
Object.entries(obj) // [["a",1],["b",2],["c",3]]
Object.assign({}, obj, { d: 4 }) // shallow merge
Object.freeze(obj)  // immutable
Object.seal(obj)     // can update values but not add/delete keys
Object.fromEntries([["a",1],["b",2]]) // {a:1, b:2}
Object.hasOwn(obj, "a") // true (modern)
"a" in obj           // true (checks prototype too)
```

 Copy

String Methods

```
const s = "Hello World";

s.length           // 11
s.toUpperCase()     // "HELLO WORLD"
s.toLowerCase()     // "hello world"
s.trim()            // remove whitespace both ends
s.trimStart()       // remove leading whitespace
s.trimEnd()         // remove trailing whitespace
s.includes("World") // true
s.startsWith("He")  // true
s.endsWith("ld")    // true
s.indexOf("o")       // 4
s.slice(0, 5)        // "Hello"
s.substring(6, 11)   // "World"
s.replace("World","JS") // "Hello JS"
s.replaceAll("l","L") // "HeLlO WorLd"
s.split(" ")        // ["Hello","World"]
s.padStart(15, "-")  // "----Hello World"
s.padEnd(15, "-")    // "Hello World----"
s.repeat(2)          // "Hello WorldHello World"
s.charAt(0)          // "H"
s.charCodeAt(0)      // 72
String.fromCharCode(72) // "H"
s.at(-1)             // "d" (last char)
```

 Copy

Math Methods

```
Math.round(4.6)    // 5
Math.ceil(4.1)     // 5
Math.floor(4.9)    // 4
Math.abs(-5)       // 5
Math.max(1,2,3)    // 3
Math.min(1,2,3)    // 1
Math.pow(2,10)     // 1024
Math.sqrt(25)      // 5
Math.random()      // 0 to 1 (exclusive)
// Random between 1-100:
Math.floor(Math.random() * 100) + 1
Math.PI            // 3.14159...
Math.trunc(4.9)    // 4 (removes decimal, no rounding)
Math.sign(-5)      // -1
```

 Copy

Number Methods

```
Number.isInteger(42)    // true
Number.isFinite(Infinity) // false
```

 Copy

```
Number.isNaN(NaN)           //true ← use this, not global isNaN
Number.parseInt("42px")      // 42
Number.parseFloat("3.14x")   // 3.14
(3.14159).toFixed(2)        // "3.14"
(1234567).toLocaleString()  // "1,234,567"
Number.MAX_SAFE_INTEGER     // 9007199254740991
```

Destructuring — Arrays & Objects

```
// Array destructuring
const [first, second, ...rest] = [10, 20, 30, 40];
// first=10, second=20, rest=[30,40]

// Swap variables
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

// Object destructuring
const { name, age, city = "Unknown" } = { name: "Ravi", age: 25 };
// city defaults to "Unknown" if not present

// Rename while destructuring
const { name: userName } = { name: "Sita" };
// userName = "Sita"

// Nested destructuring
const { address: { pincode } } = { address: { pincode: "110001" } };

// In function params
function display({ name, age }) {
  console.log(`${name} is ${age}`);
}
```

[Copy](#)

Spread & Rest Operators

 **Spread** is like opening a gift box and placing each item on the table. **Rest** is like scooping leftover candies into a bag labeled "the rest".

Spread (...) — expands

```
const a = [1, 2, 3];
const b = [...a, 4, 5]; // [1,2,3,4,5]

const obj1 = { x: 1 };
const obj2 = { ...obj1, y: 2 }; // {x:1,y:2}

// Clone an array (shallow)
const clone = [...a];

// Pass array as args
Math.max(...a); // 3
```

[Copy](#)

Rest (...) — collects

```
function sum(...nums) {
  // nums is a real array
  return nums.reduce((a, b) => a + b, 0);
}
sum(1, 2, 3, 4); // 10

// In destructuring
const [first, ...others] = [1, 2, 3, 4];
// first=1, others=[2,3,4]

// Object rest
const { a, ...remaining } = { a:1, b:2, c:3 };
// remaining = { b:2, c:3 }
```

[Copy](#)

Map vs Set vs Object vs Array

Feature	Object	Map	Array	Set
Key type	String/Symbol only	Any type	Index (number)	N/A
Order	Not guaranteed	Insertion order	Index order	Insertion order
Duplicates	No duplicate keys	No duplicate keys	Allowed	No duplicates
Size	<code>Object.keys().length</code>	<code>.size</code>	<code>.length</code>	<code>.size</code>
Iterable	Indirectly	Directly	Directly	Directly

```
// Map
const map = new Map();
map.set("key", "value");
map.set(42, "number key");
map.get("key");    // "value"
map.has("key");    // true
map.delete("key");

map.size;          // 1
for(let [k,v] of map) { ... }
```

```
// Set – unique values only
const set = new Set([1,2,2,3,3]);
// {1,2,3}
set.add(4);
set.has(1);        // true
set.delete(1);
set.size;          // 3
// Remove duplicates from array:
const unique = [...new Set([1,2,2,3])];
```

Copy

🔑 Symbol Data Type

📖 Imagine every person in the world gets a serial number at birth — even if two people have the same name, their serial numbers are unique forever. Symbol works the same way — every `Symbol()` call creates a globally unique value, even with the same description.

```
const id1 = Symbol("id");
const id2 = Symbol("id");
id1 === id2; // false – always unique!
```

```
const user = {
  name: "Arjun",
  [id1]: 123    // hidden from normal enumeration
};
```

```
Object.keys(user);    // ["name"] – Symbol not included
user[id1];            // 123
```

```
// Use case: avoid property name collisions in shared objects
// Symbol.iterator, Symbol.toPrimitive are built-in well-known symbols
```

Copy

⚡ Synchronous vs Asynchronous

📖 Synchronous is like a single cashier at a counter — serve one customer, finish, then next. Asynchronous is like ordering food at a restaurant — you order (non-blocking), get a token, and the kitchen works while you sit. When ready, they call your token (callback/promise).

Synchronous

Asynchronous

```
// Executes line by line, blocking
console.log("1");
// Heavy computation blocks here
console.log("2");
// Output: 1, 2
```

Copy

```
console.log("1");
setTimeout(() => console.log("2"), 1000);
console.log("3");
// Output: 1, 3, 2
// "2" is non-blocking - JS moves on!
```

Copy

How JS Handles Async — Event Loop

JS is **single-threaded** but uses the **Event Loop** to handle async via:
Call Stack → **Web APIs** (setTimeout, fetch, DOM events) → **Task Queue / Microtask Queue** → **Event Loop picks & pushes to stack**

```
// Phases order of execution:
// 1. Call Stack (sync code)
// 2. Microtask Queue (Promises .then, queueMicrotask)
// 3. Macrotask/Task Queue (setTimeout, setInterval, I/O)

console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// Output: A D C B
// C (microtask) runs before B (macrotask)!
```

Copy

Heap & Stack

	Call Stack	Heap
Stores	Function execution contexts, primitives	Objects, arrays, functions (reference types)
Structure	LIFO (Last In First Out)	Unstructured memory pool
Size	Limited (Stack overflow)	Large (GC managed)

📞 Callbacks → Promises → Evolution

👨‍🍳 **Callback:** "Call me when you're done cooking!" You hand the chef a note (callback) and go watch TV. The problem: if the chef needs another chef who needs another chef — that's **callback hell** (Pyramid of Doom). **Promises** fixed this by saying: "I promise to return a result (or failure) — you can chain your next step."

Callback Hell

```
// Pyramid of doom - hard to read and maintain
loginUser(function(user) {
  getUserProfile(user.id, function(profile) {
    getUserPosts(profile.id, function(posts) {
      displayPosts(posts, function() {
        // 🍷 getting deeper...
      });
    });
  });
});
```

Copy

Promises — Why they came

```
// A Promise has 3 states: pending → fulfilled | rejected

const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) resolve({ data: "✅ result" });
    else reject(new Error("❌ Failed"));
  }, 1000);
});
```

Copy

```
    }, 1000);
  });

  fetchData
    .then(result => console.log(result)) // fulfilled
    .catch(err => console.error(err))    // rejected
    .finally(() => console.log("Done")); // always runs
```

Promise Chaining

```
loginUser()
  .then(user => getUserProfile(user.id))
  .then(profile => getUserPosts(profile.id))
  .then(posts => displayPosts(posts))
  .catch(err => handleError(err));
// Flat and readable! 🍕
```

Copy

Promise Types / Static Methods

Method	Behaviour
<code>Promise.all([p1,p2])</code>	Wait for ALL. Rejects if ANY fails.
<code>Promise.allSettled([...])</code>	Wait for ALL, never rejects. Reports each result.
<code>Promise.race([...])</code>	First to settle (resolve OR reject) wins.
<code>Promise.any([...])</code>	First to RESOLVE wins. Fails only if ALL reject.
<code>Promise.resolve(val)</code>	Instantly resolved promise.
<code>Promise.reject(err)</code>	Instantly rejected promise.

🕒 async / await — Why it came after Promises

📖 Even with promise chaining, you're still writing `.then().then().then()`. **async/await** (ES2017) made async code look *synchronous* — like reading a story left to right, top to bottom. Under the hood, it's still Promises.

```
// async function always returns a Promise
async function loadDashboard() {
  try {
    const user = await loginUser();           // waits here
    const profile = await getProfile(user.id); // then here
    const posts = await getPosts(profile.id);
    displayPosts(posts);
  } catch (err) {
    console.error("Error:", err.message);
  } finally {
    hideLoader();
  }
}

// Run multiple in parallel (don't await sequentially if independent!)
const [a, b] = await Promise.all([fetchA(), fetchB()]);
// Promise.all is FASTER than awaiting one by one
```

Copy

🌐 Fetch API

```
// fetch returns a Promise that resolves to a Response object
fetch("https://api.example.com/users")
```

Copy

```
.then(res => {
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json(); // returns another Promise
})
.then(data => console.log(data))
.catch(err => console.error(err));

// With async/await (cleaner):
async function getUsers() {
  const res = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name: "Ravi" })
  });
  if (!res.ok) throw new Error("Request failed");
  return await res.json();
}
```

Events — Listener, Handler, Propagation, Delegation

 An event is like someone pressing a doorbell. The **event listener** is the wiring from the bell to the alert system. The **event handler** is the security guard who decides what to do when the bell rings.

	Event Listener	Event Handler
How	<code>addEventListener("click", fn)</code>	<code>onclick = fn</code>
Multiple	Yes — stack multiple	No — replaces previous
Remove	<code>removeEventListener</code>	Set to null

Event Propagation

```
// 3 phases:
// 1. CAPTURING: event goes DOWN from window → target
// 2. TARGET: event at the exact element
// 3. BUBBLING: event goes UP from target → window (default)

// addEventListener 3rd arg: true = capturing, false (default) = bubbling

document.getElementById("parent").addEventListener("click", fn, true); // capture
document.getElementById("child").addEventListener("click", fn); // bubble

event.stopPropagation(); // stops bubbling/capturing
event.preventDefault(); // stops default browser behavior
```

 Copy

Event Delegation

 Instead of hiring 100 security guards (one per door), hire ONE manager at the main entrance who checks the visitor's destination and routes accordingly. Event delegation puts ONE listener on the parent to handle events from all children.

```
// Instead of adding listener to each <li>:
document.getElementById("list").addEventListener("click", function(e) {
  if (e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});
// Works even for dynamically added <li> elements!
```

 Copy

DOM (Document Object Model)

Tree representation of your HTML. JS can read/modify it.

```
document.getElementById("id")
document.querySelector(".class")
document.querySelectorAll("li")

// Modify
el.textContent = "New text";
el.innerHTML = "<b>Bold</b>";
el.style.color = "red";
el.classList.add("active");
el.classList.remove("active");
el.classList.toggle("active");
el.setAttribute("href", "...");

// Create & append
const div = document.createElement("div");
document.body.appendChild(div);
div.remove();
```

BOM (Browser Object Model)

Represents the browser window — not part of the DOM.

```
window.innerWidth // viewport width
window.location.href // current URL
window.location.reload()
window.history.back()
window.history.push() // SPA routing
navigator.userAgent // browser info
navigator.geolocation
screen.width
localStorage.setItem("key", "val")
sessionStorage.setItem("key", "val")
alert("Hi");
confirm("Sure?");
prompt("Enter name:");
```

🔧 Misc: strict mode, == vs ===, CommonJS vs ES6, static

Strict Mode

```
"use strict"; // put at top of file or function

// Prevents:
x = 5; // ❌ ReferenceError – must declare with var/let/const
delete Object.prototype; // ❌ not allowed
function fn(a, a) {} // ❌ duplicate params
```

== VS ===

```
// = (loose equality) – does type coercion
0 = false; // true ← BAD
"1" = 1; // true
null = undefined; // true

// === (strict equality) – no coercion
0 === false; // false ✅
"1" === 1; // false ✅
// Always prefer ===
```

CommonJS (CJS) vs ES6 Modules

	CommonJS (Node.js)	ES6 Modules
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous (static)
Environment	Node.js (original)	Browser + modern Node
Tree-shaking	Not possible	Possible
File ext	.js (default)	.mjs or "type":"module"

Function Scope vs Block Scope

```
// var is function-scoped – visible throughout the function
function fn() {
  if(true) { var x = 1; }
  console.log(x); // 1 ← var leaks out of the block

  if(true) { let y = 2; }
  console.log(y); // ❌ ReferenceError – let is block-scoped
}
```

[Copy](#)

static keyword

```
class MathUtil {
  static PI = 3.14;           // belongs to CLASS, not instances
  static add(a, b) { return a + b; }
}

MathUtil.PI;           // 3.14 ✅
MathUtil.add(2, 3); // 5 ✅
const m = new MathUtil();
m.PI;                  // undefined – not on instance
```

[Copy](#)

🔗 Shallow Copy vs Deep Copy

📁 **Shallow copy** is like photocopying only the first page of a book — the words on page 1 are new, but the rest still refers to the original chapters. **Deep copy** makes a complete independent clone of every page.

Shallow Copy

```
const orig = { a: 1, nested: { b: 2 } };

// Methods:
const s1 = Object.assign({}, orig);
const s2 = { ...orig };

// Problem: nested objects are still shared!
s1.a = 99;           // ✅ doesn't affect orig
s1.nested.b = 99; // ❌ AFFECTS orig.nested.b!
```

[Copy](#)

Deep Copy

```
// Simple (no functions/Date/RegExp):
const deep = JSON.parse(JSON.stringify(orig));

// Modern (handles most types):
const deep2 = structuredClone(orig);

// Or use lodash _.cloneDeep(orig)

deep.nested.b = 99; // ✅ orig.nested.b unchanged
```

[Copy](#)

🕒 Debouncing & Throttling

📁 **Debounce**: An elevator door — it waits until no one is pressing the button for X seconds before closing. **Throttle**: A machine gun with a rate limiter — no matter how fast you pull the trigger, it fires at most once per second.

Debounce — wait for pause

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

// Use case: search bar – call API only after user stops typing
const search = debounce((query) => fetchResults(query), 500);
```

[Copy](#)

Throttle — limit rate

```
function throttle(fn, limit) {
  let lastCall = 0;
  return function(...args) {
    const now = Date.now();
    if (now - lastCall ≥ limit) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}
```

[Copy](#)

```
// Use case: scroll events, resize handlers
window.addEventListener("scroll", throttle(handleScroll, 200));
```

CRUD in Nested Objects & Array of Objects

```
// Initial data
let users = [
  { id: 1, name: "Arjun", address: { city: "Delhi", pin: "110001" } },
  { id: 2, name: "Priya", address: { city: "Mumbai" } }
];

// — CREATE —
users.push({ id: 3, name: "Ravi", address: { city: "Chennai" } });

// — READ —
const user = users.find(u => u.id === 1);
const city = user?.address?.city ?? "Unknown"; // optional chaining + nullish

// — UPDATE —
users = users.map(u => u.id === 1
  ? { ...u, name: "Arjun Kumar", address: { ...u.address, pin: "110002" } }
  : u
);

// — DELETE —
users = users.filter(u => u.id !== 2);

// — DEFAULT VALUES in nested —
const { name = "Guest", address: { city = "Unknown", pin = "000000" } = {} } = user || {};

// — SET value in deeply nested object —
const data = {};
// Safely set deeply nested key without error:
data.user = data.user || {};
data.user.profile = data.user.profile || {};
data.user.profile.name = "Kiran";
```

[Copy](#)

Practical: map · filter · reduce

```
const students = [
  { name: "Arjun", marks: 85, pass: true },
  { name: "Priya", marks: 42, pass: false },
  { name: "Ravi", marks: 91, pass: true },
  { name: "Meena", marks: 55, pass: true }
];

// MAP — transform each student to just their name
const names = students.map(s => s.name);
// ["Arjun","Priya","Ravi","Meena"]

// FILTER — only passed students
const passed = students.filter(s => s.pass);
// [{name:"Arjun",...},{name:"Ravi",...},{name:"Meena",...}]

// REDUCE — total marks
const total = students.reduce((acc, s) => acc + s.marks, 0);
// 273

// CHAIN — average marks of passed students
const avg = students
  .filter(s => s.pass)
  .reduce((acc, s, _, arr) => acc + s.marks / arr.length, 0);
```

[Copy](#)

```
// (85+91+55)/3 = 77
```

```
// Find highest scorer
const topScorer = students.reduce((best, s) => s.marks > best.marks ? s : best);
// {name:"Ravi", marks:91, ...}
```



Node.js

JavaScript on the server — fast, event-driven, and non-blocking

What is Node.js

 In 2009, Ryan Dahl was frustrated that Apache web servers blocked threads waiting for I/O (like a cashier freezing until the kitchen finishes). He took Chrome's V8 engine, wrapped it in C++ with libuv (an event loop library), and created **Node.js** — JavaScript outside the browser, with non-blocking I/O. The result: a single-threaded server that could handle thousands of concurrent connections.

Key Facts

- **Created:** 2009 by Ryan Dahl
- **Engine:** V8 (Google)
- **I/O:** Non-blocking via libuv
- **Threading:** Single-threaded event loop + thread pool
- **Package manager:** npm (Node Package Manager)

Features

- Non-blocking, event-driven I/O
- npm — world's largest package registry
- Single language front+back
- Streams for large data
- Great for: REST APIs, real-time apps, microservices
- NOT great for: CPU-heavy tasks

Alternatives

- Deno (Ryan Dahl again — fixes Node's regrets)
- Bun (new, faster JS runtime)
- Python (Django/Flask), Ruby (Rails), Go, Java

Architecture

Event Loop + Thread Pool: Node's main thread runs the event loop. For blocking I/O (file, DB), libuv uses a thread pool (4 threads by default). The result arrives via a callback back to the event loop — keeping the main thread free.

Thread vs Process

	Thread	Process
Memory	Shared within process	Isolated memory space
Speed	Faster to create	Slower to create
Crash	Can crash whole process	Isolated — others survive

Parallelism vs Concurrency

```
// Concurrency: juggling multiple tasks (may not be at exact same instant)
// Node.js is concurrent via event loop – one thread, many tasks in flight

// Parallelism: truly simultaneous execution on multiple cores
// Node achieves this via: cluster module, worker_threads, child_process

// Analogy:
// Concurrency = one chef multitasking (chopping while something boils)
// Parallelism = multiple chefs cooking simultaneously
```

 Copy

📦 Modules in Node.js

Type	Examples	How to use
Built-in	fs, http, path, os, events, crypto, util, cluster, child_process	<code>require('fs')</code>
Third-party	express, mongoose, lodash, axios, jsonwebtoken	Install via npm, then <code>require</code>
Custom	Your own files	<code>module.exports = ...</code> / <code>require('./myFile')</code>

```
// Custom module – math.js
function add(a, b) { return a + b; }
module.exports = { add };           // CJS

// OR ES6:
export function add(a, b) { return a + b; }

// Using it:
const { add } = require('./math'); // CJS
import { add } from './math.js';   // ES6

// Useful built-in modules:
const fs      = require('fs');      // File System
const path    = require('path');    // Path utilities
const os      = require('os');      // OS info
const events  = require('events');  // EventEmitter
const util    = require('util');    // Promisify and more
const crypto  = require('crypto');  // Hashing, encryption
const cluster = require('cluster'); // Multi-core
const { Worker } = require('worker_threads'); // Worker threads
```

Copy

REPL, npm, npx, nvm

Tool	What it does	Example
REPL	Interactive Node.js shell (Read-Eval-Print-Loop)	Just type <code>node</code> in terminal
npm	Install & manage packages	<code>npm install express</code>
npx	Execute npm packages without global install	<code>npx create-react-app</code>
nvm	Node Version Manager — switch Node versions	<code>nvm use 18</code>

package.json vs package-lock.json

```
// package.json – your project manifest
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": { "start": "node index.js", "dev": "nodemon index.js" },
  "dependencies": { "express": "^4.18.0" },    // production deps
  "devDependencies": { "nodemon": "^3.0.0" }   // dev only
}

// package-lock.json – exact versions tree (generated automatically)
// NEVER edit manually. Ensures reproducible installs across machines.
// Commit it to git!
```

Copy

.env & .gitignore

```
// .env – store secrets (never commit to git!)
PORT=3000
DB_URL=mongodb://localhost:27017/mydb
JWT_SECRET=supersecretkey

// Access in Node:
```

Copy

```
require('dotenv').config();
process.env.PORT;          // "3000"
process.env.JWT_SECRET;

// .gitignore - tell git what to ignore
node_modules/
.env
*.log
dist/
.DS_Store
```

Cluster & Event Modules

```
// Cluster - use all CPU cores
const cluster = require('cluster');
const os = require('os');
if (cluster.isPrimary) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork(); // one worker per CPU
} else {
  require('./server'); // each worker runs your server
}

// Event module - pub/sub inside Node
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.on('data', (msg) => console.log("Received:", msg));
emitter.emit('data', "Hello!");

// util module
const util = require('util');
const readFile = util.promisify(fs.readFile); // converts callback fn to Promise
const content = await readFile('file.txt', 'utf8');
```

Copy

child_process: spawn vs fork

	spawn	fork
Purpose	Run any shell command or executable	Run another Node.js script
Communication	Via streams (stdin/stdout)	Via IPC (message passing)
Example	<code>spawn('ls', ['-la'])</code>	<code>fork('./worker.js')</code>

Worker Threads vs Normal Threads

```
// Worker threads - for CPU-intensive tasks (image processing, data parsing)
// They run in separate threads BUT share memory via SharedArrayBuffer
const { Worker } = require('worker_threads');
const worker = new Worker('./heavyTask.js');
worker.on('message', result => console.log(result));

// Normal thread = the main event loop thread in Node.js
// Worker threads don't block the main thread - that's the point!
```

Copy

 HTTP vs Express — Framework vs Runtime

	http (built-in)	express (framework)
What	Core Node module	Third-party web framework
Routing	Manual (if/else)	Built-in router
Middleware	None built-in	Rich middleware system
Code	More verbose	Clean & concise

```
// Framework vs Runtime Environment:
// Runtime = environment to execute code (Node.js itself)
// Framework = built ON TOP of runtime with pre-built structure (Express)
```

 Copy

```
// http module
const http = require('http');
http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, {'Content-Type': 'text/plain'});
    res.end("Hello World");
  }
}).listen(3000);
```

```
// Express equivalent (much cleaner)
const express = require('express');
const app = express();
app.get('/', (req, res) => res.send("Hello World"));
app.listen(3000);
```

Middleware — Types & Usage

📌 Middleware is like airport security. Every passenger (request) goes through multiple checkpoints before boarding (response): ticket check → baggage scan → passport control → boarding. Each checkpoint can let you through, reject you, or redirect you.

Type	When to use	Example
Application-level	Runs for all routes	<code>app.use(fn)</code>
Router-level	Runs for specific router	<code>router.use(fn)</code>
Error-handling	Catches errors (4 params)	<code>app.use((err, req, res, next) => {})</code>
Built-in	JSON parsing, static files	<code>express.json()</code> , <code>express.static()</code>
Third-party	Extra functionality	morgan, cors, multer, helmet

```
// Morgan — HTTP request logger
const morgan = require('morgan');
app.use(morgan('dev')); // logs: GET / 200 5.123 ms

// Multer — file uploads
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });
app.post('/upload', upload.single('photo'), (req, res) => {
  res.send(req.file); // file info
});

// Express built-in
app.use(express.json()); // parse JSON bodies
app.use(express.urlencoded({ extended: true })); // parse form data
app.use(express.static('public')); // serve static files

// Error handling middleware (must be LAST, 4 params)
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: err.message });
});

// Auth middleware
function requireLogin(req, res, next) {
  if (!req.session?.userId) return res.redirect('/login');
  next();
}
app.get('/dashboard', requireLogin, (req, res) => res.render('dashboard'));
```

 Copy

📌 A template engine merges your HTML structure with dynamic data server-side. Unlike a plain HTML file with a script tag (client-side rendering), EJS runs on the server — so the data is already baked into the HTML before it arrives at the browser. Better for SEO, auth-protected pages, and MVC separation.

```
// Why not just HTML + script tags?
// - HTML + client script = browser fetches HTML, then JS, then data (slower)
// - Template engine = server sends fully-rendered HTML in one shot (faster first paint)

// EJS Syntax:
// <%= expr %>    - output escaped HTML (safe)
// <%- expr %>    - output UNescaped HTML (raw)
// <% code %>     - execute JS (no output)
// <%# comment %> - EJS comment (not in output)
// <%- include('partial.ejs') %> - include another file

// index.ejs
<h1>Welcome, <%= user.name %></h1>
<% if(isAdmin) { %>
  <a href="/admin">Admin Panel</a>
<% } %>
<ul>
<% users.forEach(u => { %>
  <li><%= u.name %></li>
<% }); %>
</ul>

// In Express:
app.set('view engine', 'ejs');
app.set('views', './views');
app.get('/', (req, res) => {
  res.render('index', { user: req.session.user, isAdmin: true });
});
```

Copy

app.locals & app.all

```
// app.locals - data available to ALL views automatically
app.locals.siteName = "MyApp";
app.locals.currentYear = new Date().getFullYear();
// In any EJS: <%= siteName %>

// app.all - matches ALL HTTP methods for a route
app.all('/secret', (req, res, next) => {
  console.log(`${req.method} request to /secret`);
  next();
});
```

Copy

🏠 Routing, Params, Router Chaining

	Path Params	Query Params
Syntax	<code>/users/:id</code>	<code>/users?search=ram&page=2</code>
Access	<code>req.params.id</code>	<code>req.query.search</code>
Used for	Required resource identifier	Optional filters, pagination, search
Example	<code>GET /users/42</code>	<code>GET /users?role=admin</code>

```
// Router chaining - clean code
const router = express.Router();

router.route('/users')
```

Copy

```
.get(getUsers)
    .post(createUser);

router.route('/users/:id')
    .get(getUser)
    .put(updateUser)
    .delete(deleteUser);

app.use('/api', router);

// Router chaining on same path — DRY!
```

 Auth, Sessions & Cookies

■ **Authentication** = "Who are you?" (proving identity — login with email+password). **Authorization** = "What are you allowed to do?" (checking permissions — admin can delete, user cannot).

Session/Cookie Story: When you log into Netflix, Netflix gives you a wristband (session ID in cookie). Every time you click, you show your wristband. Netflix checks its guestlist (session store on server) to see your name and what you can access. When you logout, Netflix crosses your name off the list.

```
// Sessions with express-session
const session = require('express-session');
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true,      // HTTPS only
    httpOnly: true,    // no JS access (prevents XSS)
    maxAge: 24 * 60 * 60 * 1000 // 1 day
  }
}));

// Login — create session
app.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user || !await bcrypt.compare(req.body.password, user.password)) {
    return res.status(401).send("Invalid credentials");
  }
  req.session.userId = user._id;
  req.session.role = user.role;
  res.redirect('/dashboard');
});

// Logout — destroy session
app.post('/logout', (req, res) => {
  req.session.destroy(() => {
    res.clearCookie('connect.sid');
    res.redirect('/login');
  });
});
```

 Copy

	Cookie	Session Storage	Local Storage
Stored	Browser (sent to server)	Browser only	Browser only
Lifetime	Set by server / expiry	Tab close	Until cleared
Size	~4KB	~5MB	~5-10MB
Server access	Yes	No	No

```
// SOP (Same-Origin Policy) – browser security: page at origin A
// cannot fetch data from origin B by default.

// CORS (Cross-Origin Resource Sharing) – server tells browser
// "I allow requests from these origins"
const cors = require('cors');
app.use(cors({ origin: 'https://myfrontend.com', credentials: true }));

// TLS/SSL – encrypts data in transit (HTTPS).
// TLS is the updated version of SSL.
// Certificate issued by a Certificate Authority (CA).
// Without it: data sent over HTTP is plain text (sniffable).
```

Security — Top 10 Vulnerabilities & Validation

Vulnerability	What	Prevention
XSS	Inject malicious scripts into pages	Escape output, CSP headers, httpOnly cookies
CSRF	Trick user's browser into making unwanted requests	CSRF tokens, SameSite cookies
SQL Injection	Inject SQL via form inputs	Parameterized queries, ORMs
SSRF	Server fetches attacker-controlled URLs	Whitelist URLs, block internal IPs
Broken Auth	Weak passwords, no rate limit	bcrypt, account lockout, MFA
Insecure Deserialization	Tampered serialized data	Validate/sign serialized objects
Security Misconfiguration	Default passwords, open ports	Harden configs, use helmet.js
Sensitive Data Exposure	Passwords in logs, no HTTPS	Encrypt at rest & transit, never log secrets
Broken Access Control	User accesses admin routes	Role-based auth middleware on every protected route
Components with Vulnerabilities	Outdated npm packages	Run <code>npm audit</code> , update regularly

Validation — Client, Server, & Pre-DB

```
// Client-side: HTML5 required, pattern, min/max + JS validation
// → UX only! NEVER trust client-side alone.

// Server-side: validate every incoming req.body
const { body, validationResult } = require('express-validator');
app.post('/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 8 }),
  body('name').trim().notEmpty()
], (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });
  // proceed to DB
});

// Pre-DB security (between server validation and DB write):
// 1. Sanitize – strip HTML tags, remove special chars (DOMPurify, sanitize-html)
// 2. Mongoose schema validation – schema.path type/min/max/enum
// 3. Schema-level unique/required constraints
// 4. Parameterized queries (Mongoose ORM handles this)
// 5. Rate limiting – prevent brute force (express-rate-limit)
const rateLimit = require('express-rate-limit');
app.use('/login', rateLimit({ windowMs: 15*60*1000, max: 10 }));
```

APIs — What, Why, Types, REST

 API (Application Programming Interface) is like a restaurant menu. You (client) don't go into the kitchen (server internals) — you tell the waiter (API) what you want, and the kitchen prepares and returns it. The menu is the contract: specific items, specific formats.

Types of APIs

- **REST API** — HTTP-based, stateless, resource-oriented (most common)
- **GraphQL** — query exactly what you need (flexible)
- **WebSocket** — persistent bi-directional connection (real-time)
- **gRPC** — high-performance, binary protocol (microservices)
- **SOAP** — XML-based, legacy enterprise

REST API Principles

- Stateless — no server-side session between requests
- Uniform interface — standard HTTP verbs (GET, POST, PUT, DELETE)
- Resource-based URLs — `/users` , `/users/42`
- JSON data exchange
- Cacheable responses
- Layered system

Is Node/Express stateless? **By default, yes** (each HTTP request is independent). BUT sessions add statefulness on top via cookies + server-side session store.

Common HTTP Status Codes

Code	Meaning	When
200	OK	Successful GET/PUT
201	Created	Successful POST (new resource)
204	No Content	Successful DELETE
301	Moved Permanently	URL has changed
302	Found (Redirect)	Temporary redirect (after login)
304	Not Modified	Cached resource still valid
400	Bad Request	Validation error, malformed input
401	Unauthorized	Not logged in
403	Forbidden	Logged in but no permission
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate entry
422	Unprocessable Entity	Validation failed
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unhandled server crash
502	Bad Gateway	Proxy/load balancer issue
503	Service Unavailable	Server down/overloaded

MVC & MVCS Architecture

```
// MVC: Model - View - Controller
// Model = Data layer (DB schema, business data)
// View = Presentation (EJS templates, HTML)
// Controller = Logic (handles req, calls model, sends response)
```

 Copy

```
// MVCS adds: Service layer (business logic separated from controller)
// Controller → Service → Model → DB
// Benefit: Controllers stay thin, Services are reusable & testable

// Flow:
// Request → Router → Middleware → Controller → Service → Model → DB
//           ← Response ← Controller ← Service ← Model ← DB result
```



MongoDB

The flexible, document-based NoSQL database

What is MongoDB — DB Types & Comparison

📖 MongoDB was created in 2007 by Eliot Horowitz & Dwight Merriman (10gen, now MongoDB Inc.) because they were frustrated building web apps on rigid SQL schemas that couldn't adapt fast enough. The name comes from "humongous" — built for massive scale. Instead of rows in tables, MongoDB stores flexible JSON-like documents.

SQL vs NoSQL

Feature	SQL (Relational)	NoSQL (Non-Relational)
Structure	Tables, rows, columns	Documents, collections, key-value, graph
Schema	Fixed, predefined	Flexible / schema-less
Scaling	Vertical (scale up)	Horizontal (scale out / sharding)
Joins	Built-in JOINS	\$lookup (aggregation) or embedding
ACID	Full ACID	BASE (usually), ACID with replica sets (MongoDB 4+)
Examples	MySQL, PostgreSQL, Oracle, SQLite	MongoDB, Redis, Cassandra, DynamoDB, Neo4j

MongoDB Architecture

```
Database → Collections → Documents → Fields (BSON)
// Like:   Database → Tables   → Rows     → Columns (SQL equivalent)

// BSON (Binary JSON) — why not plain JSON?
// BSON adds: Date type, Binary, ObjectId, more number types (Int32, Int64)
// BSON is faster to parse (binary) and richer than JSON
// MongoDB stores BSON, communicates BSON over wire

// WiredTiger — MongoDB's default storage engine (since v3.2)
// Features: compression, document-level locking, transactions, journaling
// Journaling = write-ahead log — ensures data recovery after crash
```

📄 Copy

Read/Write Path

```
// WRITE PATH:
// Client → mongod (server) → WiredTiger cache (in memory)
//           → Journal (disk — guarantees durability)
//           → Data files (disk — checkpoint every 60s)

// READ PATH:
// Client → mongod → check WiredTiger cache first
//           → if not cached → read from data files
//           → store in cache for future reads
```

📄 Copy

```
// Indexes speed up reads (pre-built lookup tables)
```

⚙️ Shell Commands, CRUD & Utility

```
// — UTILITY COMMANDS —
show dbs                // list all databases
use myDB                // switch to / create database
db.dropDatabase()       // delete current database ⚠️
show collections        // list collections in current DB
db.createCollection("users") // explicit create
db.users.drop()         // delete collection ⚠️

// Can we RENAME a database? No directly!
// Workaround: copyTo another name + drop original
// Can we RENAME a collection?
db.oldName.renameCollection("newName"); // ✅ yes!

// — CREATE —
db.users.insertOne({ name: "Arjun", age: 25, city: "Delhi" })
db.users.insertMany([ { name: "Priya" }, { name: "Ravi" } ])

// — READ —
db.users.find()          // all documents
db.users.find({ age: { $gt: 18 } }) // filter
db.users.findOne({ name: "Arjun" }) // single doc
db.users.find().limit(5).skip(10).sort({ name: 1 }) // pagination
db.users.find({}, { name: 1, _id: 0 }) // projection

// — UPDATE —
db.users.updateOne({ name: "Arjun" }, { $set: { age: 26 } })
db.users.updateMany({ city: "Delhi" }, { $set: { country: "India" } })
db.users.findOneAndUpdate({ name: "Arjun" }, { $set: { age: 27 } }, { returnDocument: "after" })
// $set — update specific field
// $unset — remove field: { $unset: { fieldName: "" } }
// $inc — increment: { $inc: { age: 1 } }
// $push — add to array: { $push: { tags: "new" } }
// $pull — remove from array: { $pull: { tags: "old" } }
// $addToSet — add to array only if unique (no duplicates)

// — DELETE —
db.users.deleteOne({ name: "Arjun" })
db.users.deleteMany({ age: { $lt: 18 } })

// — DISTINCT (unique values) —
db.users.distinct("city") // all unique city values

// — Comparison Operators —
// $eq $ne $gt $gte $lt $lte $in $nin

// — Logical Operators —
// $and $or $not $nor
```

[Copy](#)

🔗 Aggregation Pipeline

■ Think of aggregation as an assembly line. Each stage does one job and passes the result to the next. Raw documents go in, processed results come out. Stages are like stations: one filters, one groups, one sorts, one shapes the output.

```
db.orders.aggregate([
  // Stage 1: Filter
  { $match: { status: "completed", amount: { $gt: 100 } } },
```

[Copy](#)

```

// Stage 2: Group and summarize
{ $group: {
  _id: "$customerId",
  totalSpent: { $sum: "$amount" },
  orderCount: { $count: {} },
  avgOrder: { $avg: "$amount" }
}},

// Stage 3: Add computed field
{ $addFields: { isHighValue: { $gt: ["$totalSpent", 1000] } }},

// Stage 4: Sort
{ $sort: { totalSpent: -1 } },

// Stage 5: Limit
{ $limit: 10 },

// Stage 6: Shape output
{ $project: { customerId: "$_id", totalSpent: 1, orderCount: 1, _id: 0 }},

// Stage 7: Lookup (JOIN)
{ $lookup: {
  from: "customers",           // join with customers collection
  localField: "customerId",
  foreignField: "_id",
  as: "customerInfo"
}},

// Stage 8: Write to new collection
{ $out: "top_customers" }      // creates/replaces collection
});

// $expr - use aggregation expressions in $match
db.orders.find({ $expr: { $gt: ["$amount", "$budget"] } })

// $cond - conditional (like ternary)
{ $cond: { if: { $gt: ["$score", 90] }, then: "A", else: "B" } }

// $facet - multiple pipelines in one query
{ $facet: {
  byStatus: [{ $group: { _id: "$status", count: { $sum: 1 } } }],
  byCity: [{ $group: { _id: "$city", count: { $sum: 1 } } } ]
}}

// $bucket - group into ranges
{ $bucket: {
  groupBy: "$age",
  boundaries: [0, 18, 30, 60, 100],
  default: "other",
  output: { count: { $sum: 1 } }
}}

```

📁 Indexes — Types, Testing & Management

📖 An index is like a book's table of contents. Without it, MongoDB reads every page (collection scan = COLLSCAN). With an index, it jumps directly to the right page (IXSCAN). Creating the right index is the #1 performance optimization in MongoDB.

```

// Create indexes
db.users.createIndex({ name: 1 })           // ascending
db.users.createIndex({ name: -1 })          // descending
db.users.createIndex({ email: 1 }, { unique: true }) // unique index
db.users.createIndex({ name: 1, age: -1 })   // compound index
db.users.createIndex({ bio: "text" })        // text index (full-text search)
db.users.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 }) // TTL index
db.users.createIndex({ location: "2dsphere" }) // geo index

```

📋 Copy

```
db.users.createIndex({ tags: 1 }) // multikey (array) index

// Wildcard index - index all fields
db.users.createIndex({ "$*": 1 })

// List indexes
db.users.getIndexes()

// Delete an index
db.users.dropIndex("name_1")
db.users.dropIndex({ name: 1 })
db.users.dropIndexes() // drop ALL (except _id)

// Test if index is working:
db.users.find({ name: "Arjun" }).explain("executionStats")
// Look for: winningPlan.stage = "IXSCAN" ✅ (not COLLSCAN ❌)
// totalDocsExamined should be close to totalDocsReturned for efficiency

// Capped Collection - fixed-size, auto-oldest-removal (like a circular buffer)
db.createCollection("logs", { capped: true, size: 1000000, max: 1000 })
// Great for: logs, activity feeds, real-time event streams

// TTL Index - auto-delete documents after time expires
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 }) // delete after 1 hour
```

📈 Scaling, Replication & Sharding

■ **Vertical scaling** = upgrading the single server to more CPU/RAM. Like replacing a tiny engine with a bigger one. Has limits. **Horizontal scaling** = adding more servers. Like adding more cars to carry more passengers. MongoDB shines here.

	Replication	Sharding
Purpose	High availability, redundancy	Horizontal scaling for large data
How	Same data copied to multiple nodes	Data split across multiple nodes by shard key
MongoDB term	Replica Set	Sharded Cluster

```
// Replica Set: 1 Primary + N Secondaries
// - Primary handles all writes
// - Secondaries replicate from primary
// - If primary fails → election → secondary becomes primary
// - Provides read scaling (read from secondaries)
// - Minimum 3 nodes for election quorum

// Data Set = the complete dataset stored by a MongoDB instance

// Sharding: shard key determines which shard a document goes to
// config servers store cluster metadata
// mongos = query router - clients connect to mongos, not shards directly
```

📄 Copy

⚖️ ACID, BASE & CAP Theorem

ACID (SQL / MongoDB 4+)	BASE (traditional NoSQL)
Atomicity — all or nothing	Basically Available — always responds
Consistency — valid state always	Soft-state — may not be latest immediately
Isolation — concurrent txns don't interfere	Eventually Consistent — syncs over time

Durability — committed data survives crash

Trade: availability over strict consistency

CAP Theorem: A distributed system can only guarantee 2 of 3:**C**onsistency | **A**vailability | **P**artition Tolerance

MongoDB chooses CP (Consistency + Partition Tolerance) with its replica sets. During a network partition, it sacrifices availability to maintain consistency.

```
// When to use ACID: financial transactions, user records, inventory
// When to use BASE: social media feeds, logs, analytics (eventual OK)
// CAP is about distributed systems — must choose a trade-off
```

 Copy

Schema Design, Embedding vs Referencing

■ **Embedding** = put related data inside the same document (like keeping your passport in your travel bag — always together). **Referencing** = store a link (ObjectId) to another document (like a library card that points to a book on a different shelf — join needed to get it).

```
// EMBEDDING — use when:
// - Data is always read together
// - 1-to-few relationship (a user and their 3 addresses)
// - Data is unique to that parent
{
  _id: ObjectId("..."),
  name: "Arjun",
  addresses: [
    { type: "home", city: "Delhi", pin: "110001" },
    { type: "work", city: "Gurgaon", pin: "122001" }
  ]
}

// REFERENCING — use when:
// - Data is shared across many documents
// - Large nested arrays (could exceed 16MB doc limit)
// - 1-to-many / many-to-many relationships
// User document:
{ _id: ObjectId("userId1"), name: "Arjun", orderId: [ObjectId("..."), ...] }
// Order document:
{ _id: ObjectId("orderId1"), userId: ObjectId("userId1"), total: 500 }

// Database Modelling = designing how your data is structured in MongoDB
// Schema Designing = defining what fields exist, their types, validation rules
// (In Mongoose):
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  isBlocked: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});
```

 Copy

Misc: addToSet, \$out, mongodump, lookup, indexes

```
// $addToSet — add to array only if NOT already present
db.users.updateOne({ _id: id }, { $addToSet: { tags: "verified" } })
// If tags already has "verified" → nothing added (no duplicates)
// vs $push → always adds (allows duplicates)
```

```
// mongodump — backup utility
```

 Copy

```
// mongodump --uri="mongodb://localhost:27017" --db=myDB --out=/backup
// mongorestore --uri="..." --db=myDB /backup/myDB

// $lookup - LEFT OUTER JOIN equivalent
{ $lookup: {
  from: "orders",           // collection to join
  localField: "_id",        // field in current collection
  foreignField: "userId",    // field in the joined collection
  as: "userOrders"         // output array field name
}}

// How to check if index is working:
db.users.find({ name: "Arjun" }).explain("executionStats")
// → executionStats.winningPlan.stage === "IXSCAN" = good ✅
// → executionStats.nReturned vs totalDocsExamined ratio

// Check query efficiency:
db.users.find({ ... }).explain("executionStats")
// Look at: totalDocsExamined, executionTimeMillis, nReturned
```



Project Guide

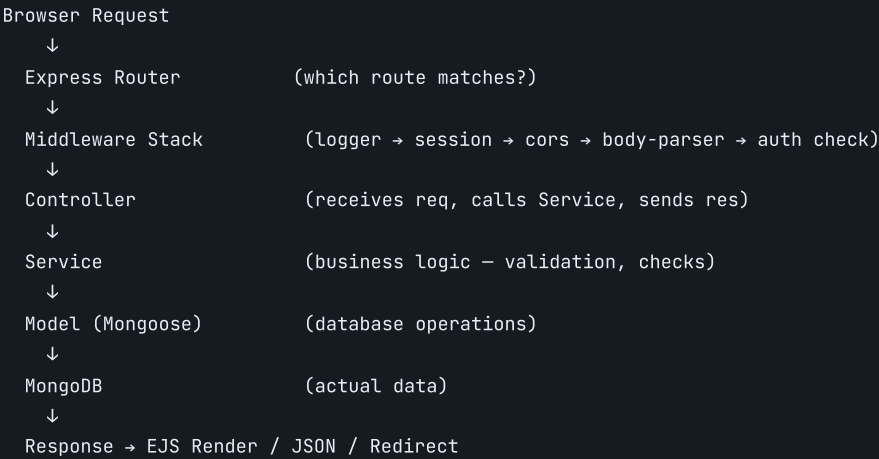
User Management System — MVCS, EJS, MongoDB, Sessions



Project Overview & Architecture

Stack: Node.js + Express + MongoDB (Mongoose) + EJS + express-session + bcrypt + express-validator + connect-mongo
Architecture: MVCS (Model → View → Controller → Service) + Session-based auth

Request Flow



Copy



Folder Structure

```
user-management/
├── .env                # secrets (never commit)
├── .gitignore
├── package.json
├── app.js              # Express app setup, middleware registration
├── server.js          # HTTP server (listens on PORT)
├──
├── config/
├── └── db.js           # Mongoose connection
```

Copy

```

├── session.js           # Session config
├── models/
│   ├── User.js         # Mongoose User schema
│   └── Admin.js        # Admin schema (or role field in User)
├── controllers/
│   ├── userController.js # signup, login, logout, profile, deleteAccount
│   └── adminController.js # dashboard, listUsers, createUser, editUser, deleteUser, blockUser
├── services/
│   ├── userService.js   # business logic for user ops
│   └── adminService.js  # business logic for admin ops
├── routes/
│   ├── userRoutes.js    # /signup /login /logout /profile
│   └── adminRoutes.js   # /admin /admin/users /admin/users/:id
├── middlewares/
│   ├── authMiddleware.js # isLoggedIn, isAdmin
│   ├── errorHandler.js  # global error middleware
│   ├── logger.js        # morgan or custom logger
│   └── cache.js         # cache-control headers
├── views/
│   ├── layouts/         # header.ejs, footer.ejs
│   ├── user/
│   │   ├── signup.ejs
│   │   ├── login.ejs
│   │   └── home.ejs     # welcome page
│   └── admin/
│       ├── login.ejs
│       ├── dashboard.ejs
│       └── users/
│           ├── list.ejs # user list with search/sort/pagination
│           ├── create.ejs
│           └── edit.ejs
├── public/              # static assets
│   ├── css/style.css
│   ├── js/main.js
│   └── uploads/         # profile images (multer)

```

⚙️ Full Workflow — Code Essentials

```

// — app.js —
const express = require('express');
const session = require('express-session');
const MongoStore = require('connect-mongo');
const morgan = require('morgan');
const path = require('path');
require('dotenv').config();

const app = express();

// Middleware
app.use(morgan('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Session
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,

```

📄 Copy

```

saveUninitialized: false,
store: MongoStore.create({ mongoUrl: process.env.DB_URL }),
cookie: { httpOnly: true, maxAge: 1000 * 60 * 60 * 24 }
}));

// Pass user to all views
app.use((req, res, next) => {
  res.locals.currentUser = req.session.user || null;
  res.locals.isAdmin = req.session.isAdmin || false;
  next();
});

// Routes
app.use('/', require('./routes/userRoutes'));
app.use('/admin', require('./routes/adminRoutes'));

// Error handler (must be last)
app.use(require('./middlewares/errorHandler'));

module.exports = app;

// — models/User.js —
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  isBlocked: { type: Boolean, default: false },
  profileImage: String,
  lastLogin: Date,
  createdAt: { type: Date, default: Date.now }
});
// Hash password before save
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
module.exports = mongoose.model('User', userSchema);

```

🔑 Admin Portal — CRUD, Search, Sort

```

// — services/adminService.js —
const User = require('../models/User');

const listUsers = async ({ search, sort, page = 1, limit = 10 }) => {
  const query = {};
  if (search) {
    query.$or = [
      { name: new RegExp(search, 'i') },
      { email: new RegExp(search, 'i') }
    ];
  }
  const sortOption = {};
  if (sort === 'asc') sortOption.name = 1;
  else if (sort === 'desc') sortOption.name = -1;

  const skip = (page - 1) * limit;
  const users = await User.find(query).sort(sortOption).skip(skip).limit(limit);
  const total = await User.countDocuments(query);
  return { users, total, pages: Math.ceil(total / limit) };
};

const createUser = async (data) => {
  const exists = await User.findOne({ email: data.email });

```

Copy

```

if (exists) throw new Error('Email already exists');
return await User.create(data);
};

const updateUser = async (id, data) => {
  return await User.findByIdAndUpdate(id, { $set: data }, { new: true, runValidators: true });
};

const deleteUser = async (id) => {
  return await User.findByIdAndDelete(id);
};

const toggleBlock = async (id) => {
  const user = await User.findById(id);
  user.isBlocked = !user.isBlocked;
  return await user.save();
};

// — views/admin/users/list.ejs (relevant snippet) —
// Search bar:
// <form method="GET" action="/admin/users">
//   <input name="search" value="<%= search %>" placeholder="Search users...">
//   <select name="sort">
//     <option value="asc">A → Z</option>
//     <option value="desc">Z → A</option>
//   </select>
//   <button type="submit">Go</button>
// </form>

```

Confirmation Modal Pattern

```

// Every destructive action shows a confirm modal
// In EJS:
// <button onClick="confirmAction('Delete user Arjun?', '/admin/users/123/delete')">Delete</button>

// In JS:
function confirmAction(message, actionUrl) {
  document.getElementById('modal-message').textContent = message;
  document.getElementById('modal-form').action = actionUrl;
  document.getElementById('confirm-modal').classList.remove('hidden');
}
// Modal has Cancel and Confirm buttons

```

Copy



User Side — Signup, Login, Home, Profile

```

// — controllers/userController.js —
const userService = require('../services/userService');

// SIGNUP
exports.signup = async (req, res, next) => {
  try {
    await userService.register(req.body);
    res.redirect('/login?msg=registered');
  } catch (err) { next(err); }
};

// LOGIN
exports.login = async (req, res, next) => {
  try {
    const user = await userService.login(req.body.email, req.body.password);
    req.session.user = { id: user._id, name: user.name, email: user.email };
    user.lastLogin = new Date();
    await user.save();
    res.redirect('/home');
  } catch (err) {
    res.render('user/login', { error: err.message });
  }
};

```

Copy

```

    }
  };

  // HOME - protected
  exports.home = (req, res) => {
    res.render('user/home', { user: req.session.user });
  };

  // PROFILE
  exports.profile = async (req, res, next) => {
    try {
      const user = await User.findById(req.session.user.id);
      res.render('user/profile', { user });
    } catch (err) { next(err); }
  };

  // DELETE ACCOUNT (with confirm modal)
  exports.deleteAccount = async (req, res, next) => {
    try {
      await User.findByIdAndDelete(req.session.user.id);
      req.session.destroy();
      res.redirect('/signup?msg=deleted');
    } catch (err) { next(err); }
  };

  // — home.ejs (snippet) —
  // <h1>Welcome, <%= user.name %>! 🍕</h1>
  // <a href="/profile">My Profile</a>
  // <form method="POST" action="/logout">
  //   <button>Logout</button>
  // </form>

  // — profile.ejs (snippet) —
  // Last login: <%= new Date(user.lastLogin).toLocaleString() %>
  // Email: <%= user.email %>
  // <button onclick="showDeleteModal()">Delete Account</button>

```